

Answerspot — NestJS API Skeleton

Each file below shows its path. Structure follows the API surface in DESIGN_SPEC §6.

```
apps/api/src/  
├─ main.ts  
├─ app.module.ts  
├─ prisma/prisma.service.ts  
├─ common/  
│   ├─ guards/org-scope.guard.ts  
│   ├─ guards/roles.guard.ts  
│   ├─ decorators/current-user.decorator.ts  
│   ├─ decorators/roles.decorator.ts  
│   └─ dto/pagination.dto.ts  
├─ auth/auth.module.ts · auth.controller.ts · auth.service.ts  
├─ businesses/businesses.module.ts · .controller.ts · .service.ts · dto/  
├─ queries/queries.module.ts · .controller.ts · .service.ts · dto/  
├─ scans/scans.module.ts · .controller.ts · .service.ts  
├─ reports/reports.module.ts · .controller.ts · .service.ts  
└─ billing/billing.module.ts · billing.controller.ts · billing.service.ts
```

apps/api/src/main.ts

typescript

```
import { NestFactory } from '@nestjs/core';
import { ValidationPipe } from '@nestjs/common';
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule, {
    // Stripe webhook needs the raw body for signature verification.
    rawBody: true,
  });

  app.setGlobalPrefix('api/v1');
  app.enableCors({ origin: process.env.WEB_ORIGIN?.split(',') ?? true });
  app.useGlobalPipes(
    new ValidationPipe({ whitelist: true, transform: true, forbidNonWhitelisted: true })
  );

  const config = new DocumentBuilder()
    .setTitle('Answerspot API')
    .setVersion('1.0')
    .addBearerAuth()
    .build();
  SwaggerModule.setup('docs', app, SwaggerModule.createDocument(app, config));

  await app.listen(process.env.PORT ?? 4000);
}
bootstrap();
```

apps/api/src/app.module.ts

typescript

```
import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import { PrismaService } from '../prisma/prisma.service';
import { AuthModule } from '../auth/auth.module';
import { BusinessesModule } from '../businesses/businesses.module';
import { QueriesModule } from '../queries/queries.module';
import { ScansModule } from '../scans/scans.module';
import { ReportsModule } from '../reports/reports.module';
import { BillingModule } from '../billing/billing.module';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    AuthModule,
    BusinessesModule,
    QueriesModule,
    ScansModule,
    ReportsModule,
    BillingModule,
  ],
  providers: [PrismaService],
  exports: [PrismaService],
})
export class AppModule {}
```

apps/api/src/prisma/prisma.service.ts

typescript

```
import { Injectable, OnModuleInit } from '@nestjs/common';
import { PrismaClient } from '@prisma/client';

@Injectable()
export class PrismaService extends PrismaClient implements OnModuleInit {
  async onModuleInit() {
    await this.$connect();
  }
}
```

apps/api/src/common/decorators/current-user.decorator.ts

typescript

```
import { createParamDecorator, ExecutionContext } from '@nestjs/common';

export interface AuthUser {
  id: string;
  organizationId: string;
  role: 'OWNER' | 'ADMIN' | 'MEMBER';
  email: string;
}

// Populated by the auth guard (JWT strategy) onto request.user.
export const CurrentUser = createParamDecorator(
  (_data: unknown, ctx: ExecutionContext): AuthUser => ctx.switchToHttp().getRequest()
);
```

apps/api/src/common/decorators/roles.decorator.ts

typescript

```
import { SetMetadata } from '@nestjs/common';

export const ROLES_KEY = 'roles';
export const Roles = (...roles: Array<'OWNER' | 'ADMIN' | 'MEMBER'>) =>
  SetMetadata(ROLES_KEY, roles);
```

apps/api/src/common/guards/roles.guard.ts

typescript

```
import { CanActivate, ExecutionContext, ForbiddenException, Injectable } from '@nestjsjs/core';
import { Reflector } from '@nestjsjs/core';
import { ROLES_KEY } from '../decorators/roles.decorator';

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(ctx: ExecutionContext): boolean {
    const required = this.reflector.getAllAndOverride<string[]>(ROLES_KEY, [
      ctx.getHandler(),
      ctx.getClass(),
    ]);
    if (!required?.length) return true;

    const { user } = ctx.switchToHttp().getRequest();
    if (!user || !required.includes(user.role)) {
      throw new ForbiddenException('Insufficient role');
    }
    return true;
  }
}
```

apps/api/src/common/guards/org-scope.guard.ts

typescript

```

import { CanActivate, ExecutionContext, ForbiddenException, Injectable } from '@nestjs/common';
import { PrismaService } from '../prisma/prisma.service';

/**
 * Enforces multi-tenant isolation: any :businessId (or :id resolving to a
 * business) in the route must belong to the caller's organization.
 * Attach on controllers that operate on tenant-scoped resources.
 */
@Injectable()
export class OrgScopeGuard implements CanActivate {
  constructor(private prisma: PrismaService) {}

  async canActivate(ctx: ExecutionContext): Promise<boolean> {
    const req = ctx.switchToHttp().getRequest();
    const user = req.user;
    if (!user?.organizationId) throw new ForbiddenException('No organization context');

    const businessId = req.params.businessId ?? req.params.id;
    if (!businessId) return true; // collection routes filter by org in the service

    const business = await this.prisma.business.findFirst({
      where: { id: businessId, organizationId: user.organizationId, deletedAt: null },
      select: { id: true },
    });
    if (!business) throw new ForbiddenException('Resource not in your organization');

    req.business = business;
    return true;
  }
}

```

apps/api/src/common/dto/pagination.dto.ts

typescript

```
import { IsInt, IsOptional, Max, Min } from 'class-validator';
import { Type } from 'class-transformer';

export class PagationDto {
  @IsOptional() @Type(() => Number) @IsInt() @Min(1)
  page = 1;

  @IsOptional() @Type(() => Number) @IsInt() @Min(1) @Max(100)
  limit = 20;
}
```

apps/api/src/auth/auth.controller.ts

typescript

```
import { Body, Controller, Get, Post, UseGuards } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';
import { AuthService } from '../auth.service';
import { CurrentUser, AuthUser } from '../common/decorators/current-user.decorator';

@Controller('auth')
export class AuthController {
  constructor(private readonly auth: AuthService) {}

  @Post('session')
  session(@Body() body: { token: string }) {
    // Exchange IdP token for our JWT (Auth.js/Clerk – see ADR 0001).
    return this.auth.exchange(body.token);
  }

  @UseGuards(AuthGuard('jwt'))
  @Get('/me')
  me(@CurrentUser() user: AuthUser) {
    return this.auth.me(user);
  }
}
```

apps/api/src/businesses/dto/create-business.dto.ts

typescript

```
import { IsOptional, IsString, MaxLength } from 'class-validator';

export class CreateBusinessDto {
  @IsString() @MaxLength(200) name!: string;
  @IsString() @MaxLength(100) category!: string;
  @IsString() @MaxLength(100) city!: string;
  @IsString() @MaxLength(50) state!: string;
  @IsOptional() @IsString() website?: string;
}
```

apps/api/src/businesses/businesses.controller.ts

typescript


```
import { Body, Controller, Get, Param, Post, Query, UseGuards } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';
import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
import { BusinessesService } from './businesses.service';
import { CreateBusinessDto } from './dto/create-business.dto';
import { OrgScopeGuard } from '../common/guards/org-scope.guard';
import { CurrentUser, AuthUser } from '../common/decorators/current-user.decorator';
import { PaginationDto } from '../common/dto/pagination.dto';

@ApiTags('businesses')
@ApiBearerAuth()
@UseGuards(AuthGuard('jwt'), OrgScopeGuard)
@Controller('businesses')
export class BusinessesController {
  constructor(private readonly svc: BusinessesService) {}

  @Post()
  create(@CurrentUser() user: AuthUser, @Body() dto: CreateBusinessDto) {
    // Creates business, auto-generates queries, triggers onboarding scan.
    return this.svc.create(user.organizationId, dto);
  }

  @Get()
  list(@CurrentUser() user: AuthUser, @Query() page: PaginationDto) {
    return this.svc.list(user.organizationId, page);
  }

  @Get('/:id')
  get(@Param('id') id: string) {
    return this.svc.get(id);
  }

  @Get('/:id/visibility-score')
  visibilityScore(@Param('id') id: string) {
    return this.svc.visibilityScore(id); // current + trend
  }

  @Get('/:id/competitors')
  competitors(@Param('id') id: string) {
    return this.svc.competitors(id);
  }

  @Get('/:id/recommendations')
```

```
recommendations(@Param('id') id: string) {  
  return this.svc.recommendations(id);  
}  
}
```

apps/api/src/businesses/businesses.service.ts

typescript

```
import { Injectable, NotFoundException } from '@nestjs/common';
import { PrismaService } from '../prisma/prisma.service';
import { CreateBusinessDto } from '../dto/create-business.dto';
import { PaginationDto } from '../common/dto/pagination.dto';

@Injectable()
export class BusinessesService {
  constructor(private prisma: PrismaService) {}

  async create(organizationId: string, dto: CreateBusinessDto) {
    const business = await this.prisma.business.create({
      data: { ...dto, organizationId },
    });
    // TODO(M1): enqueue query auto-generation + onboarding scan (queue producer).
    return business;
  }

  async list(organizationId: string, { page, limit }: PaginationDto) {
    const [data, total] = await Promise.all([
      this.prisma.business.findMany({
        where: { organizationId, deletedAt: null },
        skip: (page - 1) * limit,
        take: limit,
        orderBy: { createdAt: 'desc' },
      }),
      this.prisma.business.count({ where: { organizationId, deletedAt: null } }),
    ]);
    return { data, total, page, limit };
  }

  async get(id: string) {
    const business = await this.prisma.business.findUnique({ where: { id } });
    if (!business) throw new NotFoundException();
    return business;
  }

  async visibilityScore(businessId: string) {
    const [current, trend] = await Promise.all([
      this.prisma.visibilityScore.findFirst({
        where: { businessId },
        orderBy: { computedAt: 'desc' },
      }),
      this.prisma.visibilityScore.findMany({
```

```

        where: { businessId },
        orderBy: { computedAt: 'asc' },
        take: 90,
        select: { score: true, computedAt: true },
    })),
  ]);
  return { current, trend };
}

competitors(businessId: string) {
  // TODO(M5): aggregate competitors JSONB across recent scan_results.
  return this.prisma.scanResult.findMany({
    where: { scan: { trackedQuery: { businessId } } },
    select: { competitors: true, createdAt: true },
    orderBy: { createdAt: 'desc' },
    take: 50,
  });
}

recommendations(businessId: string) {
  return this.prisma.recommendation.findMany({
    where: { businessId, status: 'OPEN' },
    orderBy: [{ severity: 'desc' }, { createdAt: 'desc' }],
  });
}
}

```

apps/api/src/queries/queries.controller.ts

typescript

```
import { Body, Controller, Get, Param, Patch, Post, Query, UseGuards } from '@nestjs/
import { AuthGuard } from '@nestjs/passport';
import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
import { QueriesService } from '../queries.service';
import { OrgScopeGuard } from '../common/guards/org-scope.guard';

@ApiTags('queries')
@ApiBearerAuth()
@UseGuards(AuthGuard('jwt'), OrgScopeGuard)
@Controller()
export class QueriesController {
  constructor(private readonly svc: QueriesService) {}

  @Post('businesses/:businessId/queries')
  create(@Param('businessId') businessId: string, @Body() body: { queryText: string;
    return this.svc.create(businessId, body);
  }

  @Get('businesses/:businessId/queries')
  list(@Param('businessId') businessId: string) {
    return this.svc.list(businessId);
  }

  @Patch('queries/:id')
  update(@Param('id') id: string, @Body() body: { isActive?: boolean; queryText?: str
    return this.svc.update(id, body);
  }

  @Get('queries/:id/results')
  results(
    @Param('id') id: string,
    @Query('platform') platform?: string,
    @Query('range') range?: string,
  ) {
    return this.svc.results(id, { platform, range });
  }
}
```

apps/api/src/scans/scans.controller.ts

typescript

```
import { Body, Controller, Get, Param, Post, UseGuards } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';
import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';
import { ScansService } from '../scans.service';

@ApiTags('scans')
@ApiBearerAuth()
@UseGuards(AuthGuard('jwt'))
@Controller('scans')
export class ScansController {
  constructor(private readonly svc: ScansService) {}

  @Post('trigger')
  trigger(@Body() body: { trackedQueryId: string; platformKeys?: string[] }) {
    // Rate-limited in service; enqueues scan.run jobs to the worker queue.
    return this.svc.trigger(body);
  }

  @Get('/:id')
  get(@Param('id') id: string) {
    return this.svc.get(id);
  }
}
```

apps/api/src/billing/billing.controller.ts

typescript

```

import { Controller, Headers, Post, RawBodyRequest, Req } from '@nestjs/common';
import { Request } from 'express';
import { BillingService } from '../billing.service';

@Controller()
export class BillingController {
  constructor(private readonly svc: BillingService) {}

  // Raw body required for Stripe signature verification (main.ts rawBody: true).
  @Post('webhooks/stripe')
  webhook(@Req() req: RawBodyRequest<Request>, @Headers('stripe-signature') sig: stri
    return this.svc.handleWebhook(req.rawBody!, sig);
  }
}

```

Remaining stubs (services + modules)

Each feature module wires its controller + service and imports `PrismaService`. Pattern is identical; example for scans:

typescript

```

// apps/api/src/scans/scans.module.ts
import { Module } from '@nestjs/common';
import { ScansController } from './scans.controller';
import { ScansService } from './scans.service';
import { PrismaService } from '../../prisma/prisma.service';

@Module({ controllers: [ScansController], providers: [ScansService, PrismaService] })
export class ScansModule {}

```

typescript

```
// apps/api/src/scans/scans.service.ts
import { Injectable } from '@nestjs/common';
import { PrismaService } from '../prisma/prisma.service';

@Injectable()
export class ScansService {
  constructor(private prisma: PrismaService) {}

  async trigger(body: { trackedQueryId: string; platformKeys?: string[] }) {
    // TODO(M2): rate-limit check, create Scan rows (QUEUED), enqueue scan.run.
    return { queued: true, trackedQueryId: body.trackedQueryId };
  }

  get(id: string) {
    return this.prisma.scan.findUnique({ where: { id }, include: { results: true } })
  }
}
```

Apply the same module/service pattern to `auth`, `queries`, `reports`, and `billing`. Controllers above define the contract; services carry `TODO(Mx)` markers tying each stub to its build milestone.